

Федеральное государственное автономное образовательное учреждение высшего
образования

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет информационных технологий

Кафедра «Информатика и информационные технологии»

Направление подготовки/ специальность: информационные технологии

ОТЧЕТ

по проектной практике

Студент: Лазовский Максим Олегович Группа: 251-338

Место прохождения практики: Московский Политех, кафедра информатика и
информационные технологии

Отчет принят с оценкой _____ Дата _____

Руководитель практики: _____

Москва 2026

Оглавление

ВВЕДЕНИЕ	3
1. Общая информация о проекте:.....	3
2. Общая характеристика деятельности организации (<i>заказчика проекта</i>)	3
3. Описание задания по проектной практике.....	4
Задания основной части проектной практики:	4
Задания вариативной части проектной практики:	4
Основная часть проектной практики:.....	4
Вариативная часть проектной практики:	5
ЗАКЛЮЧЕНИЕ	6
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ.....	7
ПРИЛОЖЕНИЕ 1	7
ПРИЛОЖЕНИЕ 2	10

ВВЕДЕНИЕ

Российский игровой рынок испытывает недостаток качественных отечественных проектов, способных конкурировать с зарубежными аналогами. Наш проект направлен на создание уникальной игры, которая не только заполнит эту нишу, но и станет примером успешного импортозамещения в индустрии видеоигр.

1. Общая информация о проекте:

Название проекта: группа проектов игровой индустрии Ascended

Цель: Разработать игру в жанре FPS roguelite с киберпанк-сеттингом, которая продемонстрирует потенциал российских разработчиков.

Задачи:

- 1.Формирование команды проекта и распределение ролей с учётом нового состава разработчиков;
- 2.Создание технического задания и планирование задач;
- 3.Разработка концепт-артов персонажей и локаций;
- 4.Создание систем прокачки персонажа, боевой системы и прочих механик;
- 5.Создание моделей персонажей, объектов окружения, оружия и локаций;
- 6.Создание анимаций персонажей;
- 7.Разработка UI/UX интерфейса.

2. Общая характеристика деятельности организации (*заказчика проекта*)

Наименование заказчика

Центр проектной деятельности Мосполитеха (ЦПД)

Организационная структура

Заказчик является координатором проектной деятельности. Контроль над работой осуществляет руководитель практики и куратор по <<Проектной деятельности>>. В процессе работы взаимодействие с заказчиком необходимо для получения обратной связи.

Описание деятельности

Деятельность заказчика – формирование необходим для работы над проектом компетенций.

3. Описание задания по проектной практике

Задание по проектной практике состояло из двух частей: основной и вариативной

Задания основной части проектной практики:

1. Создание и настройка git репозитория на Github/Gitverse
2. Написание документов в Markdown
3. Создание статического веб-сайта (html/css) с информацией, содержащей описание проекта, участников, журнала работы, ресурсов.
4. Взаимодействие с организацией-партнёром.
5. Составление отчёта о практике.

Задания вариативной части проектной практики:

1. Создание игры на языке Python с использованием библиотеки Pygame.
 2. Разработать основную тематику игры: RPG
 3. Проработка основных элементов: игрок, противник, NPC (взаимодействие персонажа), покупка предметов, подборка предметов, инвентарь, система боя, интерфейс, диалоги.
 4. Подготовить отчёт в формате Markdown
- #### 4. Описание достигнутых результатов по проектной практике

Были полностью выполнены все задачи основной и вариативной части проектной практики.

Основная часть проектной практики:

1. Был создан и настроен репозиторий (Лазовский Максим Олегович)
2. Разработан сайт на языках разметки (HTML/CSS), который содержит всю необходимую информацию: Главная страница (Лазовский Максим Олегович), страница <<О проекте>> (Коновалов Иван Павлович), страница <<участники>> (Лазовский Максим Олегович), страница <<журнал>> (Коновалов Иван Павлович), страница <<ресурсы>> (Лазовский Максим Олегович).

3. Создан отчёт о посещении дня карьеры <<Ростелеком>> (Коновалов Иван Павлович)
4. Изучение и создание отчёта в формате Markdown

Вариативная часть проектной практики:

1. Проработка идеи и определение жанра (Коновалов Иван Павлович)
2. Создание компонентов (Лазовский Максим Олегович 50% от всей работы)

Player – Хранит здоровье, золото, очки опыта, зелья. Атака, получение урона, лечение, повышение уровня.

Enemy – Враги, имеющие свои очки здоровья, урон, награду в виде золота и очков опыта.

NPC – неигровые персонажи, Elder-мудрец (диалог, постановка задачи игроку), Merchant-торговец (главный источник зелий лечения в игре), Healer-лекарь (источник лечения вне боя).

Item – Предметы, которые может использовать персонаж (зелья, монеты)

DialogBox – окно диалога с персонажами

VictoryScreen – экран победы (завершение поставленной задачи)

DefeatScreen – экран поражения (смерть персонажа/провал поставленной задачи)

Создание класса Game и CombatSystem и level_up (Коновалов Иван Павлович 50% всей работы)

Game – Создание окна, загрузка фона, врагов, NPC, предметов.

Управление движением, боем, диалогом, инвентарём.

CombatSystem – Боевая система: выбор из 3 возможных действий за ход (атака, сильная атака, лечение), противники наносят урон.

Level_up – система поднятия уровня, при победе над врагом персонаж получает определённое количество опыта и поднимает свой уровень, при повышении уровня персонаж полностью излечивает свои очки здоровья, а также получает бонусы к характеристикам: +25 здоровья, +15 очков маны, +4 урона, +2 снижение урона по персонажу.

ЗАКЛЮЧЕНИЕ

В результате выполнения проектной практики была сделана полноценная игра в жанре RPG на языке Python с использованием библиотеки Pygame.

В игре разработано 4 класса: Player (игрок), Enemy (противники с которыми нужно сражаться), Item (предмет, используемый персонажем), NPC (неигровые персонажи, с которыми можно взаимодействовать). Всего в игре 12 существ из них 1 игрок, 3 NPC, 8 противников.

В игре разработана полностью рабочая пошаговая боевая система, за 1 ход игрок может совершить 3 действия – атака, сильная атака (требует ману), лечение (требует зелье лечения), после каждого хода игра следует ход противника, противник может только наносить урон игроку. После победы над врагом игроку даётся определённое количество очков опыта, необходимых для повышения уровня, при повышении уровня у персонажа восполняется здоровье и количество маны, а также он получает бонусы к характеристикам (смотрите выше), у персонажа есть показатели урона, брони, здоровья, количества маны, золота, количество зелий.

Фактором победы является уничтожение всех 8 противников, при успехе, высветится соответствующее окно победы, при поражении, соответственно окно поражения.

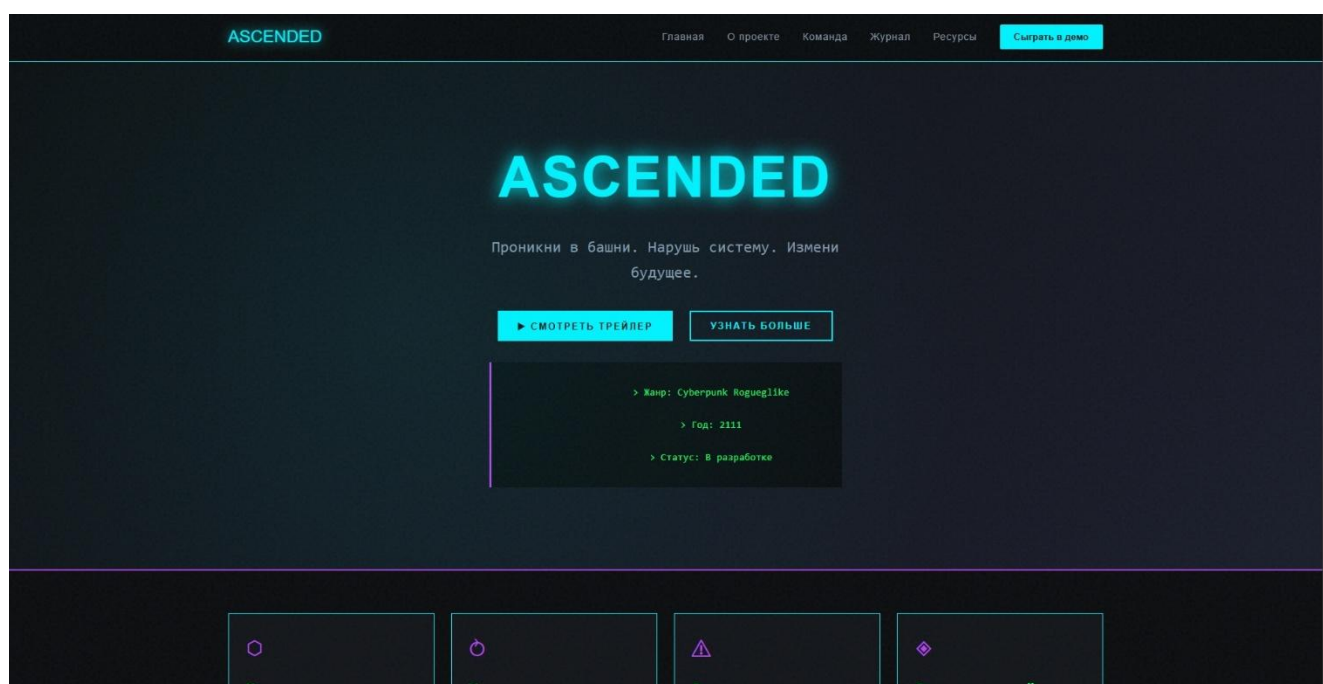
В результате работы над проектной практикой был создан статический сайт проекта по <<проектной деятельности>>, он содержит самую важную информацию: главная страница, страница о проекте, страница участников, журнал работ, страница с ресурсами.

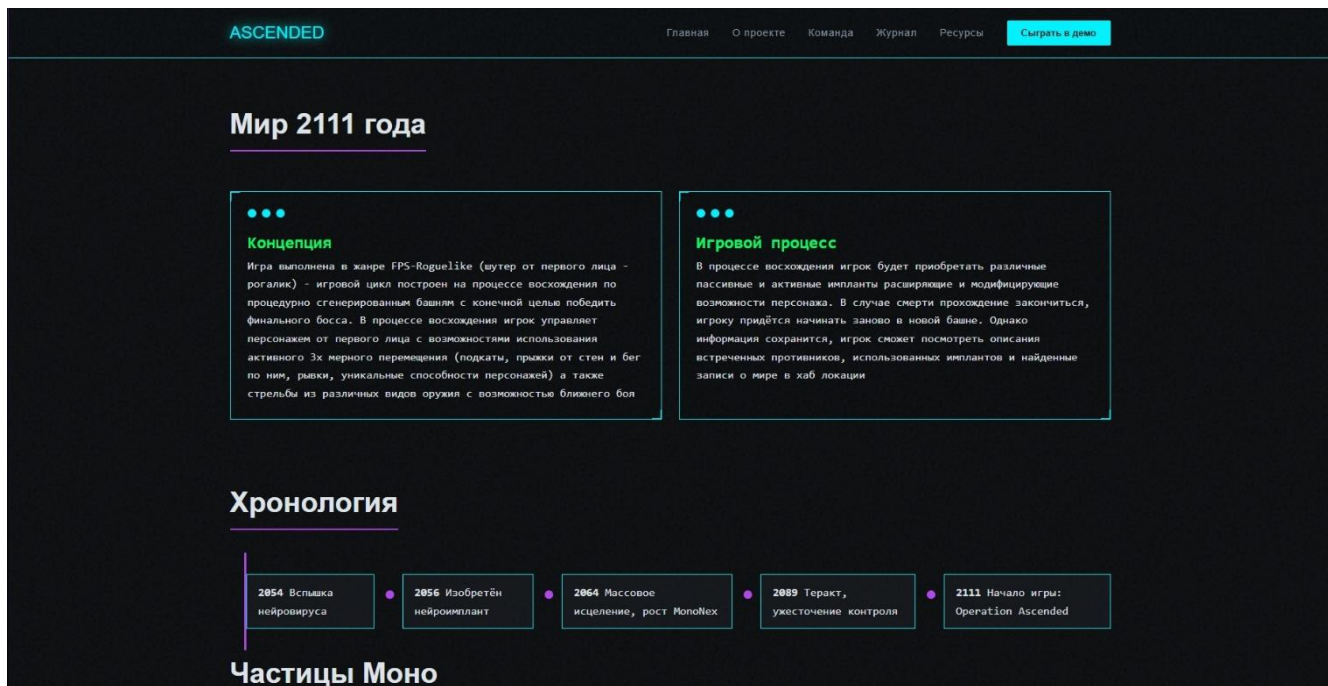
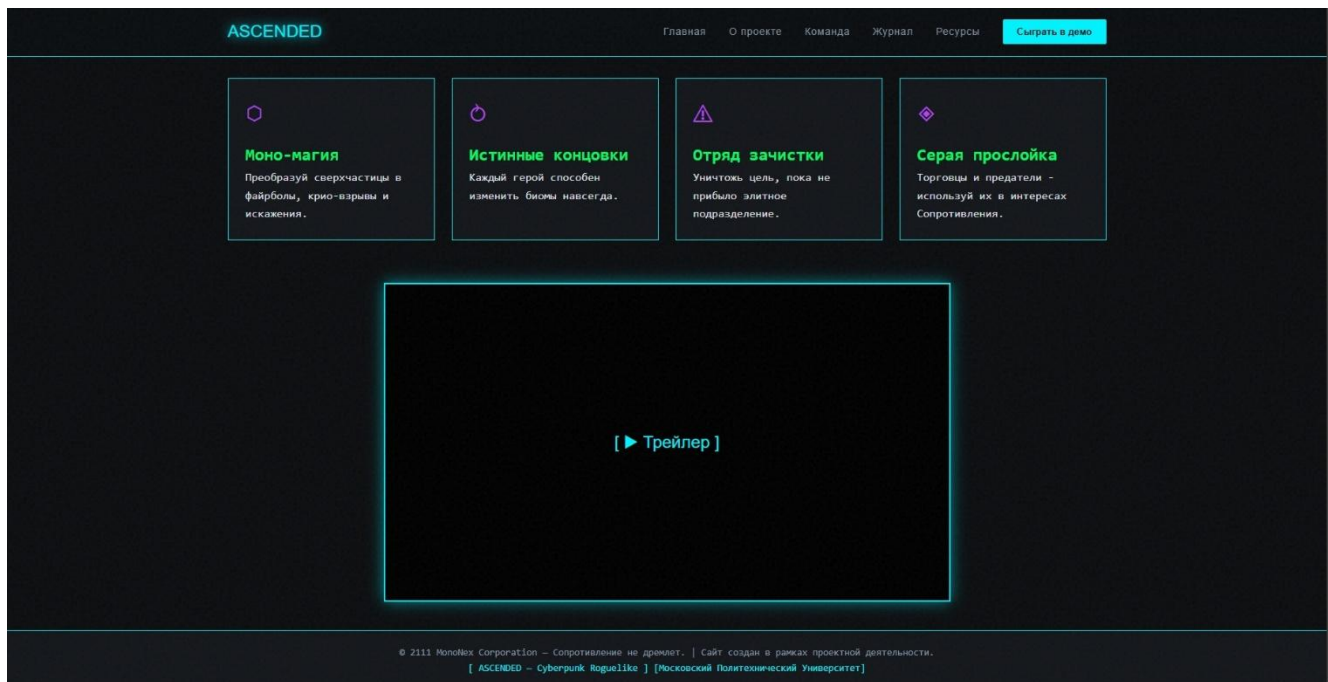
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

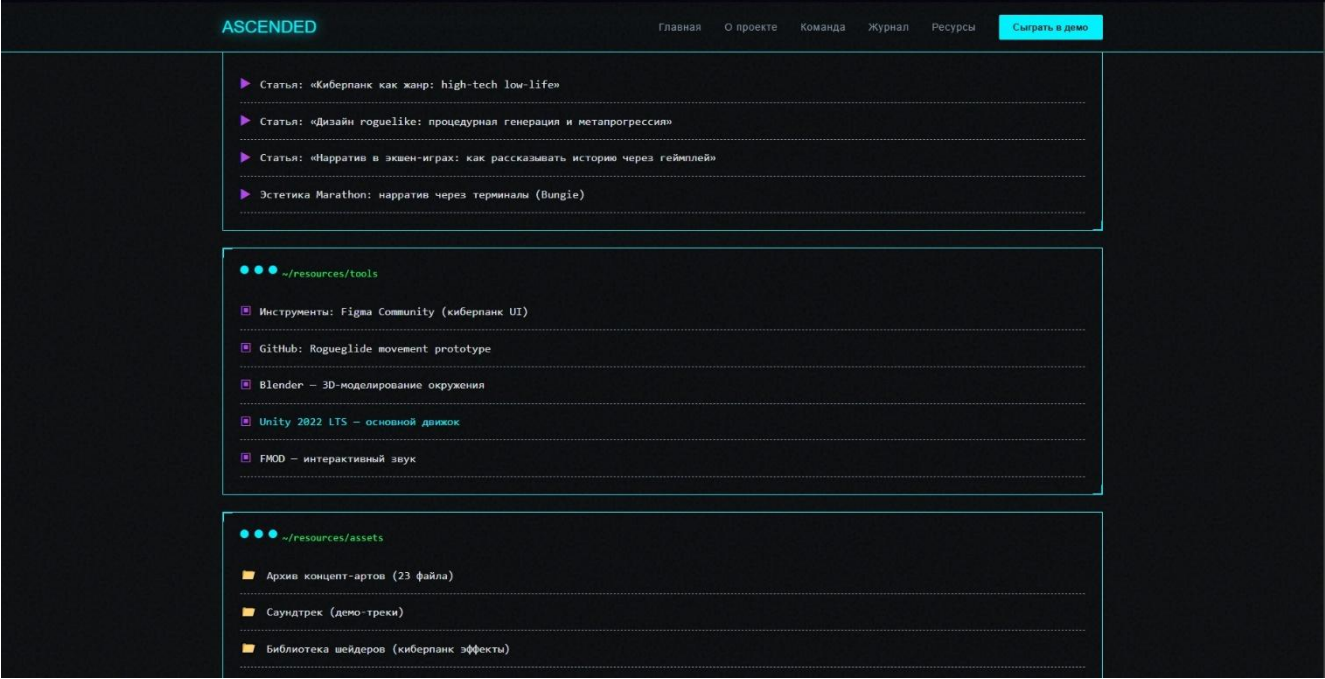
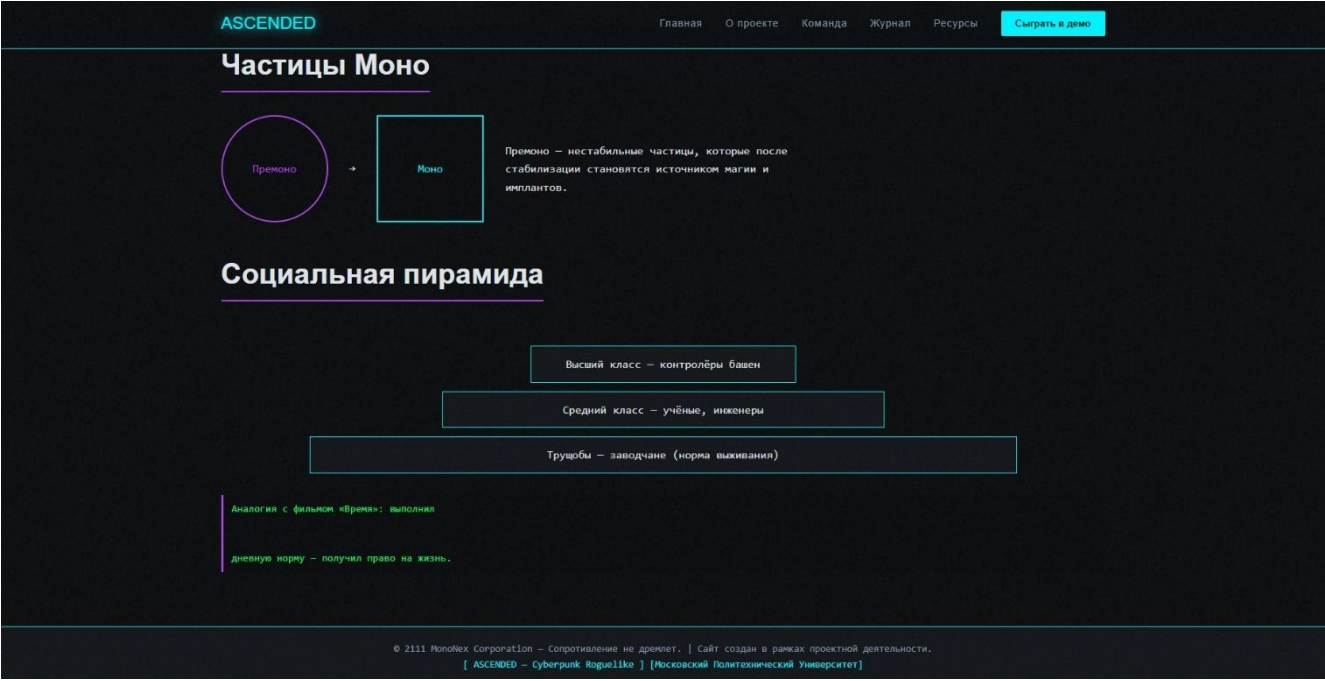
1. Руководство по Markdown – Текст: электронный // [RU.HEXLET.IO](https://ru.hexlet.io): [сайт]. – URL: <https://ru.hexlet.io/blog/posts/что-такое-markdown-i-zachem-on-nuzhen> (дата обращения: 20.05.2026)
2. Документация Pygame – Текст: электронный // [PYGAME.ORG](https://www.pygame.org): [сайт]. – URL: <https://www.pygame.org/docs/> (дата обращения: 20.05.2026)

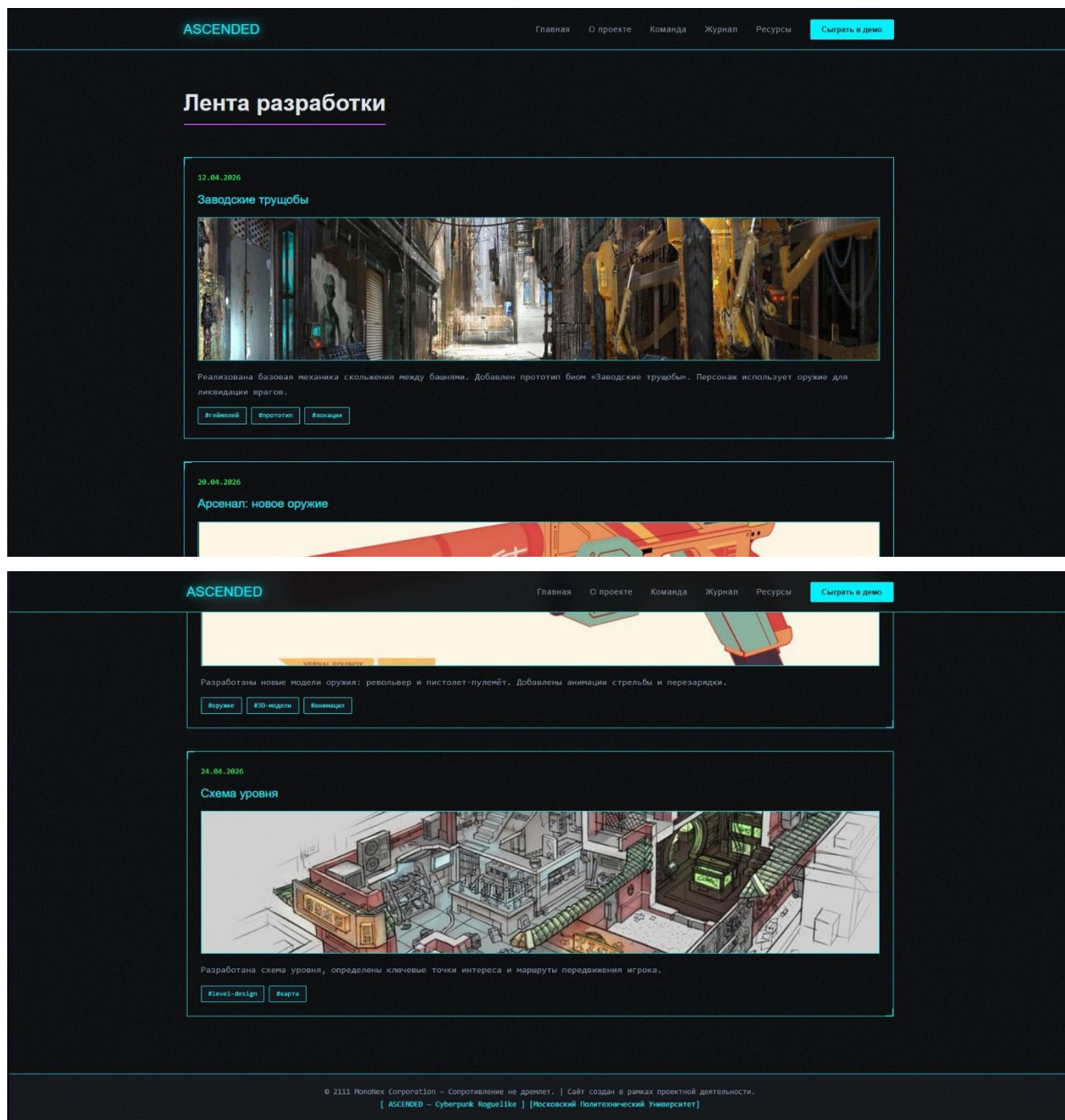
ПРИЛОЖЕНИЕ 1

Ссылка на гитхаб: <https://github.com/wassupwassup/practice-2026>









ПРИЛОЖЕНИЕ 2

```

import subprocess
import sys
import os

def install_dependencies():
    try:
        import pygame
        return True
    except ImportError:
        print(" Устанавливаем pygame...")
        try:
            subprocess.check_call([sys.executable, "-m", "pip", "install", "pygame"])
            print("pygame установлен")
            return True
        except Exception as e:
            print(f" Ошибка: {e}")
            return False

if not install_dependencies():
    input("Нажмите Enter...")
    sys.exit(1)

import pygame
import random
import math
from pygame.locals import *

# Инициализация
pygame.init()

```

Константы

SCREEN_WIDTH = 1024

SCREEN_HEIGHT = 768

FPS = 60

размеры спрайтов

SPRITE_SIZE_MAP = 96 # Размер спрайтов на карте

SPRITE_SIZE_COMBAT = 192 # Размер спрайтов в бою

ITEM_SIZE = 48 # Размер предметов

фон

BACKGROUND_IMAGE = "map.png"

USE_CUSTOM_BACKGROUND = True

фон битвы

COMBAT_BACKGROUND_IMAGE = "combat_bg.png"

USE_COMBAT_BACKGROUND = True

кастомные спрайты

PLAYER_SPRITE_PATH = "player.png"

ENEMY_SPRITES = {

'skeleton': "skeleton.png",

'ghoul': "ghoul.png",

'lich': "lich.png",

}

NPC_SPRITES = {

'merchant': "merchant.png",

'healer': "healer.png",

'elder': "elder.png"

}

Цвета

```
COLORS = {  
'BG': (20, 24, 35),  
'UI': (40, 45, 68),  
'UI_HOVER': (60, 68, 102),  
'TEXT': (220, 220, 255),  
'TEXT_DARK': (150, 160, 200),  
'HEALTH': (235, 70, 90),  
'HEALTH_BG': (70, 30, 40),  
'MANA': (70, 130, 235),  
'MANA_BG': (30, 40, 70),  
'EXP': (250, 210, 70),  
'EXP_BG': (70, 60, 40),  
'VICTORY': (100, 200, 100),  
'DEFEAT': (150, 50, 50),  
}
```

```
class ParticleSystem:
```

```
    def __init__(self):
```

```
        self.particles = []
```

```
    def emit(self, x, y, color, count=10):
```

```
        for _ in range(count):
```

```
            self.particles.append({
```

```
                'x': x, 'y': y,
```

```
                'vx': random.uniform(-100, 100),
```

```
                'vy': random.uniform(-100, 100),
```

```
                'life': 1.0,
```

```
                'color': color
```

```
})
```

```
def update(self, dt):  
    for p in self.particles[:]:  
        p['x'] += p['vx'] * dt  
        p['y'] += p['vy'] * dt  
        p['life'] -= dt * 2  
        if p['life'] <= 0:  
            self.particles.remove(p)
```

```
def draw(self, screen):  
    for p in self.particles:  
        size = int(6 * p['life'])  
        pygame.draw.circle(screen, p['color'][:3], (int(p['x']), int(p['y'])), max(1, size))
```

```
def load_sprite(path, size):  
    if path and os.path.exists(path):  
        try:  
            sprite = pygame.image.load(path).convert_alpha()  
            return pygame.transform.scale(sprite, size)  
        except Exception as e:  
            print(f" Не удалось загрузить {path}: {e}")
```

```
    raise FileNotFoundError(f" нет спрайта: {path}")
```

```
class Player:  
    def __init__(self, x, y):  
        self.x = x  
        self.y = y  
        self.name = "Герой"
```

```

self.level = 1
self.max_hp = 120
self.hp = 120
self.max_mp = 80
self.mp = 80
self.strength = 15
self.defense = 10
self.exp = 0
self.exp_to_next = 100
self.gold = 500
self.potions = 5
self.equipment = {'weapon': None, 'armor': None}
self.rect = pygame.Rect(x, y, SPRITE_SIZE_MAP, SPRITE_SIZE_MAP)

self.sprite = load_sprite(PLAYER_SPRITE_PATH, (SPRITE_SIZE_MAP,
SPRITE_SIZE_MAP))
self.combat_sprite = load_sprite(PLAYER_SPRITE_PATH,
(SPRITE_SIZE_COMBAT, SPRITE_SIZE_COMBAT))

self.move_speed = 300

def get_damage(self):
    dmg = self.strength
    if self.equipment['weapon']:
        dmg += self.equipment['weapon'].get('damage', 0)
    return dmg

def take_damage(self, dmg):
    actual = max(1, dmg - self.defense)
    self.hp -= actual

```

```
if self.hp < 0:  
    self.hp = 0  
    return actual
```

```
def heal(self, amount):  
    self.hp =
```



```
min(self.max_hp, self.hp + amount)
```

```
def is_alive(self):
```

```
    return self.hp > 0
```

```
def gain_exp(self, amount):
```

```
    self.exp += amount
```

```
    if self.exp >= self.exp_to_next:
```

```
        self.level_up()
```

```
def level_up(self):
```

```
    self.level += 1
```

```
    self.exp -= self.exp_to_next
```

```
    self.exp_to_next = int(self.exp_to_next * 1.2)
```

```
    self.max_hp += 25
```

```
    self.hp = self.max_hp
```

```
    self.max_mp += 15
```

```
    self.mp = self.max_mp
```

```
    self.strength += 4
```

```
    self.defense += 2
```

```
class Enemy:
```

```
    def __init__(self, x, y, enemy_type, level=1):
```

```
        self.x = x
```

```
        self.y = y
```

```
        self.type = enemy_type
```

```
        self.level = level
```

```
types = {
```

```
    'skeleton': {'name': 'Скелет', 'hp': 65, 'damage': 18, 'exp': 55, 'gold': 35},
```

```
'ghoul': {'name': 'Мертвец', 'hp': 333, 'damage': 15, 'exp': 75, 'gold': 10},  
'lich': {'name': 'Лич', 'hp': 666, 'damage': 33, 'exp': 777, 'gold': 1000},  
}
```

```
info = types.get(enemy_type, types['skeleton'])  
self.name = info['name']  
self.max_hp = info['hp'] + (level - 1) * 15  
self.hp = self.max_hp  
self.damage = info['damage'] + (level - 1) * 3  
self.exp_reward = info['exp'] + (level - 1) * 10  
self.gold_reward = info['gold'] + (level - 1) * 5  
self.rect = pygame.Rect(x, y, SPRITE_SIZE_MAP, SPRITE_SIZE_MAP)
```

```
self.sprite = load_sprite(ENEMY_SPRITES[enemy_type],  
                           (SPRITE_SIZE_MAP, SPRITE_SIZE_MAP))  
self.combat_sprite = load_sprite(ENEMY_SPRITES[enemy_type],  
                                  (SPRITE_SIZE_COMBAT, SPRITE_SIZE_COMBAT))
```

```
def attack(self):  
    return random.randint(self.damage - 5, self.damage + 5)
```

```
def take_damage(self, dmg):  
    self.hp -= dmg  
    if self.hp < 0:  
        self.hp = 0  
    return dmg
```

```
def is_alive(self):  
    return self.hp > 0
```

```

class NPC:
    def __init__(self, x, y, npc_id):
        self.x = x
        self.y = y
        self.id = npc_id
        self.rect = pygame.Rect(x, y, SPRITE_SIZE_MAP, SPRITE_SIZE_MAP)
        self.dialogues = self.get_dialogues()
        self.current_dialogue = 0

    def get_dialogues(self):
        dialogues = {
            'elder': ["Приветствую, славный воин!", "Нам нужна твоя помощь!", "Вокруг
много опасных монстров.",
            "Нужно очистить лес от этой нежити!"],
            'merchant': ["Добро пожаловать!", "Могу продать зелья за 30 золота.",
            "Нажми Е, чтобы купить."],
            'healer': ["Я могу исцелить тебя.", "Лечение стоит 20 золота.", "Хочешь
восстановить здоровье?"]
        }
        return dialogues.get(self.id, ["Привет!", "Чем могу помочь?"])

    def get_next_dialogue(self):
        text = self.dialogues[self.current_dialogue]
        self.current_dialogue = (self.current_dialogue + 1) % len(self.dialogues)
        return text

class Item:

```

```

def __init__(self, x, y, item_type):
    self.x = x
    self.y = y
    self.type = item_type
    self.rect = pygame.Rect(x, y, ITEM_SIZE, ITEM_SIZE)
    self.sprite = self.create_sprite()

    if item_type == 'potion':
        self.name = "Зелье здоровья"
        self.value = 50
    elif item_type == 'gold':
        self.name = "Золото"
        self.value = random.randint(10, 50)
    elif item_type == 'sword':
        self.name = "Стальной меч"
        self.value = {"damage": 12}

    def create_sprite(self):
        sprite = pygame.Surface((ITEM_SIZE, ITEM_SIZE), pygame.SRCALPHA)
        s = ITEM_SIZE

        if self.type == 'potion':
            pygame.draw.rect(sprite, (255, 50, 50), (s // 4, s // 4, s // 2, s // 1.6))
            pygame.draw.circle(sprite, (255, 50, 50), (s // 2, s // 5), s // 8)
            pygame.draw.rect(sprite, (200, 200, 200), (s // 2.3, s // 4, s // 8, s // 1.6))
        elif self.type == 'gold':

```

```

pygame.draw.ellipse(sprite, (255, 215, 0), (s // 4, s // 3, s // 2, s // 4))
pygame.draw.circle(sprite, (255, 215, 0), (s // 2, s // 2), s // 5)
elif self.type == 'sword':
    pygame.draw.line(sprite, (200, 200, 250), (s // 1.3, s // 4), (s // 4, s // 1.3), 5)
    pygame.draw.rect(sprite, (150, 100, 50), (s // 2.7, s // 1.7, s // 4, s // 4))

return sprite

```

```

class DialogueBox:
    def __init__(self, screen):
        self.screen = screen
        self.active = False
        self.lines = []
        self.current_line = 0
        self.display_text = ""
        self.char_index = 0
        self.char_timer = 0
        self.font = pygame.font.Font(None, 28)

```

```

    def start(self, dialogues):
        self.active = True
        self.lines = dialogues
        self.current_line = 0
        self.display_text = ""
        self.char_index = 0
        self.next\_line\(\)

```

```

    def next_line(self):
        if self.current_line < len(self.lines):
            self.display_text = ""

```

```

self.char_index = 0
self.target_text = self.lines[self.current_line]
self.current_line += 1
else:
    self.active = False

def update(self, dt):
    if not self.active:
        return

    self.char_timer += dt
    if self.char_timer > 0.03 and self.char_index < len(self.target_text):
        self.char_timer = 0
        self.char_index += 1
        self.display_text = self.target_text[:self.char_index]

def handle_event(self, event):
    if not self.active:
        return False

    if event.type == KEYDOWN and event.key == K_SPACE:
        if self.char_index < len(self.target_text):
            self.char_index = len(self.target_text)
            self.display_text = self.target_text
        else:
            self.next_line()
        return True
    return False

def draw(self):

```

```
if not self.active:
```

```
    return
```

```
    dialog_rect = pygame.Rect(50, SCREEN_HEIGHT - 180, SCREEN_WIDTH -  
    100, 140)
```

```
    pygame.draw.rect(self.screen, COLORS['UI'], dialog_rect)
```

```
    pygame.draw.rect(self.screen, COLORS['TEXT_DARK'], dialog_rect, 3)
```

```
    words = self.display_text.split(' ')
```

```
    lines = []
```

```
    current_line = ""
```

```
    for word in words:
```

```
        test_line = current_line + " " + word if current_line else word
```

```
        text_surface = self.font.render(test_line, True, COLORS['TEXT'])
```

```
        if text_surface.get_width() < SCREEN_WIDTH - 120:
```

```
            current_line = test_line
```

```
        else:
```

```
            if current_line:
```

```
                lines.append(current_line)
```

```
            current_line = word
```

```
    if current_line:
```

```
        lines.append(current_line)
```

```
    for i, line in enumerate(lines[:3]):
```

```
        text_surface = self.font.render(line, True, COLORS['TEXT'])
```

```
        self.screen.blit(text_surface, (70, SCREEN_HEIGHT - 160 + i * 35))
```

```
    if self.char_index >= len(self.target\_text):
```

```
hint = self.font.render("ИПОБЕЖ", True, COLORS['TEXT_DARK'])
self.screen.blit(hint, (SCREEN_WIDTH - 120, SCREEN_HEIGHT - 165))
```

```
class CombatSystem:
```

```
def __init__(self, screen, player, enemy, particles):
```

```
self.screen = screen
```

```
self.player = player
```

```
self.enemy = enemy
```

```
self.particles = particles
```

```
self.player_turn = True
```

```
self.message = ""
```

```
self.message_timer = 0
```

```
self.selected_action = 0
```

```
self.combat_active = True
```

```
self.font = pygame.font.Font(None, 36)
```

```
self.small_font = pygame.font.Font(None, 24)
```

```
self.combat_bg = self.load_combat_background()
```

```
def load_combat_background(self):
```

```
if USE_COMBAT_BACKGROUND and
```

```
os.path.exists(COMBAT_BACKGROUND_IMAGE):
```

```
try:
```

```
bg = pygame.image.load(COMBAT_BACKGROUND_IMAGE).convert()
```

```
bg = pygame.transform.scale(bg, (SCREEN_WIDTH,
```



```

SCREEN_HEIGHT))

print(f" Загружен фон битвы: {COMBAT_BACKGROUND_IMAGE}")

return bg

except Exception as e:

print(f" Не удалось загрузить фон битвы: {e}")


bg = pygame.Surface((SCREEN_WIDTH, SCREEN_HEIGHT))


if self.enemy.name == "Лич":
    for y in range(SCREEN_HEIGHT):
        r = 40 + int((y / SCREEN_HEIGHT) * 50)
        g = 20 + int((y / SCREEN_HEIGHT) * 20)
        b = 50 + int((y / SCREEN_HEIGHT) * 50)
        pygame.draw.line(bg, (r, g, b), (0, y), (SCREEN_WIDTH, y))
    else:
        for y in range(SCREEN_HEIGHT):
            color_value = int(30 + (y / SCREEN_HEIGHT) * 50)
            pygame.draw.line(bg, (color_value, color_value - 10, color_value + 20), (0, y),
                             (SCREEN_WIDTH, y))


    for _ in range(100):
        x = random.randint(0, SCREEN_WIDTH)
        y = random.randint(0, SCREEN_HEIGHT)
        brightness = random.randint(50, 150)
        pygame.draw.circle(bg, (brightness, brightness, brightness), (x, y),
                             random.randint(1, 2))


    return bg


def handle_event(self, event):

```

```

if not self.combat_active:
    return

if event.type == KEYDOWN and self.player_turn:
    if event.key == K_UP:
        self.selected_action = (self.selected_action - 1) % 3
    elif event.key == K_DOWN:
        self.selected_action = (self.selected_action + 1) % 3
    elif event.key == K_RETURN or event.key == K_SPACE:
        if self.selected_action == 0:
            self.player_attack()
        elif self.selected_action == 1:
            self.player_skill()
        elif self.selected_action == 2:
            self.player_heal()

def update(self, dt):
    if self.message_timer > 0:
        self.message_timer -= dt

if not self.player_turn and self.enemy.is\_alive\(\) and self.player.is\_alive\(\):
    self.enemy_attack()
    self.player_turn = True

if not self.enemy.is\_alive\(\) or not self.player.is\_alive\(\):
    self.combat_active = False
    return True
    return False

def player_attack(self):

```

```
damage = self.player.get_damage()
actual = self.enemy.take_damage(damage)
self.message = f" Вы нанесли {actual} урона!"
self.message_timer = 1.5
self.particles.emit(self.enemy.x + SPRITE_SIZE_MAP // 2, self.enemy.y +
SPRITE_SIZE_MAP // 2, COLORS['HEALTH'],
15)
self.player_turn = False
```

```
def player_skill(self):
if self.player.mp >= 15:
damage = self.player.get_damage() * 2
actual = self.enemy.take_damage(damage)
self.player.mp -= 15
self.message = f" Мощный удар! {actual} урона!"
self.particles.emit(self.enemy.x + SPRITE_SIZE_MAP // 2, self.enemy.y +
SPRITE_SIZE_MAP // 2,
(255, 100, 100), 25)
else:
self.message = " Недостаточно маны!"
self.message_timer = 1.0
return
self.player_turn = False
```

```
def player_heal(self):
if self.player.potions > 0:
heal_amount = 50
self.player.heal(heal_amount)
self.player.potions -= 1
self.message = f" Вы восстановили {heal_amount} HP!"
```

```
self.particles.emit(self.player.x + SPRITE_SIZE_MAP // 2, self.player.y +
SPRITE_SIZE_MAP // 2,
COLORS['HEALTH'], 20)
else:
self.message = " Нет зелий!"
self.message_timer = 1.0
return
self.player_turn = False

def enemy_attack(self):
damage = self.enemy.attack()
actual = self.player.take_damage(damage)
self.message = f" {self.enemy.name} нанес {actual} урона!"
self.message_timer = 1.5
self.particles.emit(self.player.x +
```

```
SPRITE_SIZE_MAP // 2, self.player.y + SPRITE_SIZE_MAP // 2,  
COLORS['HEALTH'], 15)
```

```
def draw(self):
```

```
self.screen.blit(self.combat_bg, (0, 0))
```

```
frame_rect = pygame.Rect(15, 15, SCREEN_WIDTH - 30, SCREEN_HEIGHT -  
30)
```

```
pygame.draw.rect(self.screen, COLORS['UI'], frame_rect, 3)
```

```
info_panel = pygame.Rect(25, 25, SCREEN_WIDTH - 50, 65)
```

```
info_surface = pygame.Surface((info_panel.width, info_panel.height),  
pygame.SRCALPHA)
```

```
info_surface.fill((0, 0, 0, 180))
```

```
self.screen.blit(info_surface, info_panel)
```

```
pygame.draw.rect(self.screen, COLORS['UI_HOVER'], info_panel, 2)
```

```
hp_text = self.small_font.render(f" {self.player.hp}/{self.player.max_hp}", True,  
COLORS['TEXT'])
```

```
self.screen.blit(hp_text, (40, 42))
```

```
mp_text = self.small_font.render(f" {self.player.mp}/{self.player.max_mp}",  
True, COLORS['TEXT'])
```

```
self.screen.blit(mp_text, (250, 42))
```

```
level_text = self.small_font.render(f" Yp. {self.player.level}", True,  
COLORS['TEXT'])
```

```
self.screen.blit(level_text, (450, 42))
```

```
enemy_info = self.small_font.render(f" {self.enemy.name", True,
```

```
COLORS['HEALTH'])
```

```
self.screen.blit(enemy_info, (SCREEN_WIDTH - 280, 42))
```

```
exp_percent = self.player.exp / self.player.exp_to_next if self.player.exp_to_next  
> 0 else 0
```

```
pygame.draw.rect(self.screen, COLORS['EXP_BG'], (40, 70, 300, 8))
```

```
pygame.draw.rect(self.screen, COLORS['EXP'], (40, 70, 300 * exp_percent, 8))
```

```
player_x = 80
```

```
enemy_x = SCREEN_WIDTH - 80 - SPRITE_SIZE_COMBAT
```

```
center_y = SCREEN_HEIGHT // 2 - SPRITE_SIZE_COMBAT // 2
```

```
player_platform = pygame.Rect(player_x - 15, center_y - 15,
```

```
SPRITE_SIZE_COMBAT + 30, SPRITE_SIZE_COMBAT + 30)
```

```
platform_surface = pygame.Surface((player_platform.width,
```

```
player_platform.height), pygame.SRCALPHA)
```

```
platform_surface.fill((0, 0, 0, 150))
```

```
self.screen.blit(platform_surface, player_platform)
```

```
pygame.draw.rect(self.screen, COLORS['UI_HOVER'], player_platform, 2)
```

```
self.screen.blit(self.player.combat_sprite, (player_x, center_y))
```

```
enemy_platform = pygame.Rect(enemy_x - 15, center_y - 15,
```

```
SPRITE_SIZE_COMBAT + 30, SPRITE_SIZE_COMBAT + 30)
```

```
platform_surface2 = pygame.Surface((enemy_platform.width,
```

```
enemy_platform.height), pygame.SRCALPHA)
```

```
platform_surface2.fill((0, 0, 0, 150))
```

```
self.screen.blit(platform_surface2, enemy_platform)
```

```
pygame.draw.rect(self.screen, COLORS['UI_HOVER'], enemy_platform, 2)
```

```
self.screen.blit(self.enemy.combat_sprite, (enemy_x, center_y))
```

```

self.draw_bar(player_x, center_y + SPRITE_SIZE_COMBAT + 10,
self.player.hp, self.player.max_hp,
COLORS['HEALTH'], COLORS['HEALTH_BG'], SPRITE_SIZE_COMBAT)
self.draw_small_bar(player_x, center_y + SPRITE_SIZE_COMBAT + 35,
self.player.mp, self.player.max_mp,
COLORS['MANA'], COLORS['MANA_BG'], SPRITE_SIZE_COMBAT)
self.draw_bar(enemy_x, center_y + SPRITE_SIZE_COMBAT + 10,
self.enemy.hp, self.enemy.max_hp, COLORS['HEALTH'],
COLORS['HEALTH_BG'], SPRITE_SIZE_COMBAT)

name_text = self.font.render(self.player.name, True, COLORS['TEXT'])
name_rect = name_text.get_rect(center=(player_x + SPRITE_SIZE_COMBAT //
2, center_y - 25))
self.screen.blit(name_text, name_rect)

enemy_name = self.font.render(self.enemy.name, True, COLORS['HEALTH'])
enemy_name_rect = enemy_name.get_rect(center=(enemy_x +
SPRITE_SIZE_COMBAT // 2, center_y - 25))
self.screen.blit(enemy_name, enemy_name_rect)

actions = [" АТАКОВАТЬ", " СИЛЬНЫЙ УДАР", " ЗЕЛЬЕ"]
menu_y = SCREEN_HEIGHT - 140

menu_panel = pygame.Rect(25, menu_y - 10, SCREEN_WIDTH - 50, 100)
menu_surface = pygame.Surface((menu_panel.width, menu_panel.height),
pygame.SRCALPHA)
menu_surface.fill((0, 0, 0, 200))
self.screen.blit(menu_surface, menu_panel)

```

```

pygame.draw.rect(self.screen, COLORS['UI_HOVER'], menu_panel, 2)

for i, action in enumerate(actions):
    x = 50 + i * 280
    color = COLORS['UI_HOVER'] if i == self.selected_action else COLORS['UI']
    button_rect = pygame.Rect(x, menu_y, 270, 50)

    if i == self.selected_action:
        for offset in range(3):
            glow_rect = button_rect.inflate(offset * 4, offset * 4)
            pygame.draw.rect(self.screen,
                             (COLORS['EXP'][0], COLORS['EXP'][1], COLORS['EXP'][2], 50 - offset * 15),
                             glow_rect, 2)

        pygame.draw.rect(self.screen, color, button_rect)
        pygame.draw.rect(self.screen, COLORS['TEXT_DARK'], button_rect, 2)

        action_text = self.font.render(action, True, COLORS['TEXT'])
        text_rect = action_text.get_rect(center=(x + 135, menu_y + 25))
        self.screen.blit(action_text, text_rect)

    if self.message_timer > 0:
        msg_bg = pygame.Rect(SCREEN_WIDTH // 2 - 200, SCREEN_HEIGHT - 220,
                              400, 40)
        msg_surface = pygame.Surface((msg_bg.width, msg_bg.height),
                                     pygame.SRCALPHA)
        msg_surface.fill((0, 0, 0, 200))
        self.screen.blit(msg_surface, msg_bg)
        pygame.draw.rect(self.screen, COLORS['EXP'], msg_bg, 2)
        msg_text = self.small_font.render(self.message, True, COLORS['EXP'])

```



```
msg_rect = msg_text.get_rect(center=(SCREEN_WIDTH // 2,  
SCREEN_HEIGHT - 200))  
self.screen.blit(msg_text, msg_rect)
```

```
hint = self.small_font.render("стрелка вверх\вниз выбор | ENTER действие",  
True, COLORS['TEXT_DARK'])  
hint_rect = hint.get_rect(center=(SCREEN_WIDTH // 2, menu_y + 75))  
self.screen.blit(hint, hint_rect)
```

```
def draw_bar(self, x, y, current, maximum, color, bg_color, width):
```

```
height = 16
```

```
percent = current / maximum if maximum > 0 else 0
```

```
pygame.draw.rect(self.screen, bg_color, (x, y, width, height))
```

```
pygame.draw.rect(self.screen, bg_color, (x, y, width, height), 1)
```

```
pygame.draw.rect(self.screen, color, (x + 1, y + 1, (width - 2) * percent, height -  
2))
```

```
font = pygame.font.Font(None, 16)
```

```
text = font.render(f"{int(current)}/{int(maximum)}", True, COLORS['TEXT'])
```

```
text_rect = text.get_rect(center=(x + width // 2, y + height // 2))
```

```
self.screen.blit(text, text_rect)
```

```
def draw_small_bar(self, x, y, current, maximum, color, bg_color, width):
```

```
height = 6
```

```
percent = current / maximum if maximum > 0 else 0
```

```
pygame.draw.rect(self.screen, bg_color, (x, y, width, height))
```

```
pygame.draw.rect(self.screen, color, (x, y, width * percent, height))
```

```
pygame.draw.rect(self.screen, COLORS['TEXT_DARK'], (x, y, width, height),
```

1)

```
class VictoryScreen:
    def __init__(self, screen):
        self.screen = screen
        self.active = False
        self.timer = 0
        self.font_big = pygame.font.Font(None, 72)
        self.font_small = pygame.font.Font(None, 36)
        self.particles = ParticleSystem()

    def start(self):
        self.active = True
        self.timer = 5.0
        for _ in range(100):
            self.particles.emit(random.randint(0, SCREEN_WIDTH), random.randint(0,
SCREEN_HEIGHT), COLORS['VICTORY'], 1)

    def update(self, dt):
        if not self.active:
            return False

        self.timer -= dt
        self.particles.update(dt)

        if self.timer <= 0:
            self.active = False
            return True
        return False
```

```
def draw(self):  
    if not self.active:  
        return  
  
    overlay = pygame.Surface((SCREEN_WIDTH, SCREEN_HEIGHT))  
    overlay.set_alpha(200)  
    overlay.fill(COLORS['BG'])  
    self.screen.blit(overlay, (0, 0))  
  
    victory_text = self.font_big.render("ПОБЕДА!", True, COLORS['VICTORY'])  
    text_rect = victory_text.get_rect(center=(SCREEN_WIDTH // 2,
```

```

SCREEN_HEIGHT // 2 - 50))
self.screen.blit(victory_text, text_rect)

congrats_text = self.font_small.render("Вы очистили эти земли от монстров!",
True, COLORS['TEXT'])
congrats_rect = congrats_text.get_rect(center=(SCREEN_WIDTH // 2,
SCREEN_HEIGHT // 2 + 20))
self.screen.blit(congrats_text, congrats_rect)

timer_text = self.font_small.render(f"Выход через {int(self.timer)}...", True,
COLORS['TEXT_DARK'])
timer_rect = timer_text.get_rect(center=(SCREEN_WIDTH // 2,
SCREEN_HEIGHT - 100))
self.screen.blit(timer_text, timer_rect)

self.particles.draw(self.screen)

class DefeatScreen:
def __init__(self, screen):
self.screen = screen
self.active = False
self.timer = 0
self.font_big = pygame.font.Font(None, 80)
self.font_medium = pygame.font.Font(None, 48)
self.font_small = pygame.font.Font(None, 32)
self.particles = ParticleSystem()

def start(self):
self.active = True
self.timer = 5.0

```

```

for _ in range(150):
self.particles.emit(random.randint(0, SCREEN_WIDTH), random.randint(0,
SCREEN_HEIGHT), COLORS['DEFEAT'], 1)

def update(self, dt):
if not self.active:
return False

self.timer -= dt
self.particles.update(dt)

if self.timer <= 0:
self.active = False
return True
return False

def draw(self):
if not self.active:
return

overlay = pygame.Surface((SCREEN_WIDTH, SCREEN_HEIGHT))
overlay.set_alpha(200)
overlay.fill(COLORS['BG'])
self.screen.blit(overlay, (0, 0))

defeat_text = self.font_big.render("ПОПАЖЕНИЕ", True, COLORS['DEFEAT'])
text_rect = defeat_text.get_rect(center=(SCREEN_WIDTH // 2,
SCREEN_HEIGHT // 2 - 80))
self.screen.blit(defeat_text, text_rect)

```

```

message_text = self.font_medium.render("Вы пали в бою...", True,
COLORS['TEXT'])
message_rect = message_text.get_rect(center=(SCREEN_WIDTH // 2,
SCREEN_HEIGHT // 2))
self.screen.blit(message_text, message_rect)

timer_text = self.font_small.render(f"Выход через {int(self.timer)}...", True,
COLORS['TEXT_DARK'])
timer_rect = timer_text.get_rect(center=(SCREEN_WIDTH // 2,
SCREEN_HEIGHT - 100))
self.screen.blit(timer_text, timer_rect)

self.particles.draw(self.screen)

class Game:
def __init__(self):
self.screen = pygame.display.set_mode((SCREEN_WIDTH,
SCREEN_HEIGHT))
pygame.display.set_caption("Изгнание нежити")
self.clock = pygame.time.Clock()
self.running = True
self.dt = 0
self.victory = False
self.defeat = False

self.background = self.load_background()
self.player = Player(SCREEN_WIDTH // 2, SCREEN_HEIGHT // 2)
self.obstacles = []
self.enemies = self.create_enemies()
self.npcs = self.create_npcs()

```

```
self.items = self.create_items()
```

```
self.particles = ParticleSystem()
```

```
self.combat = None
```

```
self.dialogue = None
```

```
self.victory_screen = VictoryScreen(self.screen)
```

```
self.defeat_screen = DefeatScreen(self.screen)
```

```
self.show_inventory = False
```

```
self.interact_cooldown = 0
```

```
def load_background(self):
```

```
if USE_CUSTOM_BACKGROUND:
```

```
try:
```

```
if 'BACKGROUND_IMAGE' in globals():
```

```
bg = pygame.image.load(BACKGROUND_IMAGE).convert()
```

```
bg = pygame.transform.scale(bg, (SCREEN_WIDTH, SCREEN_HEIGHT))
```

```
print(f" Загружен фон: {BACKGROUND_IMAGE}")
```

```
return bg
```

```
except Exception as e:
```

```
print(f" Не удалось загрузить фон: {e}")
```

```
bg = pygame.Surface((SCREEN_WIDTH, SCREEN_HEIGHT))
```

```
for y in range(SCREEN_HEIGHT):
```

```
color_value = int(20 + (y / SCREEN_HEIGHT) * 60)
```

```
pygame.draw.line(bg, (color_value, color_value +
```

```
10, color_value + 20), (0, y), (SCREEN_WIDTH, y))  
return bg
```

```
def create_enemies(self):  
    return [  
        Enemy(300, 200, 'skeleton', 2),  
        Enemy(750, 410, 'skeleton', 2),  
        Enemy(670, 350, 'skeleton', 2),  
        Enemy(750, 290, 'skeleton', 2),  
        Enemy(300, 300, 'ghoul', 2),  
        Enemy(666, 450, 'ghoul', 1),  
        Enemy(666, 250, 'ghoul', 1),  
        Enemy(900, 350, 'lich', 1),  
    ]
```

```
def create_npcs(self):  
    return [  
        NPC(70, 190, 'merchant'),  
        NPC(100, 300, 'elder'),  
        NPC(50, 379, 'healer'),  
    ]
```

```
def create_items(self):  
    return [  
        Item(350, 400, 'potion'),  
        Item(650, 200, 'gold'),  
        Item(250, 400, 'potion'),  
        Item(550, 350, 'gold'),  
    ]
```



```

def check_victory(self):
    if len(self.enemies) == 0 and not self.victory and not self.defeat:
        self.victory = True
        self.victory_screen.start()
    return True
    return False

```

```

def handle_events(self):
    for event in pygame.event.get():
        if event.type == QUIT:
            self.running = False

```

```

    if self.dialogue:
        self.dialogue.handle_event(event)
    if not self.dialogue.active:
        self.dialogue = None
    continue

```

```

    if self.combat:
        self.combat.handle_event(event)
    continue

```

```

    if event.type == KEYDOWN:
        if event.key == K_ESCAPE:
            self.running = False
        elif event.key == K_i:
            self.show_inventory = not self.show_inventory
        elif event.key == K_e and self.interact_cooldown <= 0:
            self.interact()

```

```

def interact(self):
    for npc in self.npcs:
        if self.player.rect.colliderect(npc.rect):
            if npc.id == 'merchant' and self.interact_cooldown <= 0:
                if self.player.gold >= 30:
                    self.player.gold -= 30
                    self.player.potions += 1
                    self.dialogue = DialogueBox(self.screen)
                    self.dialogue.start(["Вы купили зелье здоровья!", "Спасибо за покупку!"])
                    self.interact_cooldown = 0.5
                else:
                    self.dialogue = DialogueBox(self.screen)
                    self.dialogue.start(["Не хватает золота!", "Приходи, когда разбогатеешь."])
                    self.interact_cooldown = 0.5
            elif npc.id == 'healer' and self.interact_cooldown <= 0:
                if self.player.gold >= 20 and self.player.hp < self.player.max_hp:
                    self.player.gold -= 20
                    self.player.hp = self.player.max_hp
                    self.dialogue = DialogueBox(self.screen)
                    self.dialogue.start(["Вы восстановили здоровье!", "Будь осторожен!"])
                    self.interact_cooldown = 0.5
                elif self.player.hp >= self.player.max_hp:
                    self.dialogue = DialogueBox(self.screen)
                    self.dialogue.start(["Ты уже здоров!", "Приходи, если поранишься."])
                    self.interact_cooldown = 0.5
            else:
                self.dialogue = DialogueBox(self.screen)
                self.dialogue.start(["Не хватает золота!", "Приходи позже."])
                self.interact_cooldown = 0.5
            elif npc.id == 'elder':

```

```
self.dialogue = DialogueBox(self.screen)
self.dialogue.start(npc.dialogues)
self.interact_cooldown = 0.5
return
```

```
def update(self):
    if self.interact_cooldown > 0:
        self.interact_cooldown -= self.dt
```

```
# Обновление экранов победы/поражения
if
```

```

self.victory:
victory_end = self.victory_screen.update(self.dt)
if victory_end:
self.running = False
return

if self.defeat:
defeat_end = self.defeat_screen.update(self.dt)
if defeat_end:
self.running = False
return

if self.combat is None and self.dialogue is None and not self.show\_inventory:
keys = pygame.key.get_pressed()
dx = dy = 0

if keys[K_LEFT] or keys[K_a]: dx = -1
if keys[K_RIGHT] or keys[K_d]: dx = 1
if keys[K_UP] or keys[K_w]: dy = -1
if keys[K_DOWN] or keys[K_s]: dy = 1

if dx != 0 or dy != 0:
length = math.sqrt(dx * dx + dy * dy)
dx /= length
dy /= length

new_x = self.player.x + dx * self.player.move_speed * self.dt
new_y = self.player.y + dy * self.player.move_speed * self.dt

if 0 <= new_x <= SCREEN_WIDTH - SPRITE_SIZE_MAP and 0 <= new_y <=

```

SCREEN_HEIGHT - SPRITE_SIZE_MAP:

self.player.x = new_x

self.player.y = new_y

self.player.rect.x = self.player.x

self.player.rect.y = self.player.y

self.particles.update(self.dt)

if self.combat is None:

for enemy in self.enemies[:]:

if self.player.rect.colliderect(enemy.rect):

self.combat = CombatSystem(self.screen, self.player, enemy, self.particles)

break

if self.combat:

combat_end = self.combat.update(self.dt)

if combat_end:

if not [self.player.is_alive\(\)](#):

self.defeat = True

self.defeat_screen.start()

elif not [self.combat.enemy.is_alive\(\)](#):

self.player.gain_exp(self.combat.enemy.exp_reward)

[self.player.gold](#) += [self.combat.enemy.gold](#)_reward

if self.combat.enemy in self.enemies:

self.enemies.remove(self.combat.enemy)

self.check_victory()

self.combat = None

if self.dialogue:

self.dialogue.update(self.dt)

```

for item in self.items[:]:
    if self.player.rect.colliderect(item.rect):
        if item.type == 'potion':
            self.player.potions += 1
            self.particles.emit(item.x + ITEM_SIZE // 2, item.y + ITEM_SIZE // 2,
                                COLORS['HEALTH'], 15)
        elif item.type == 'gold':
            self.player.gold += item.value
            self.particles.emit(item.x + ITEM_SIZE // 2, item.y + ITEM_SIZE // 2, (255,
            215, 0), 15)
        elif item.type == 'sword':
            self.player.equipment['weapon'] = {'name': 'Стальной меч', 'damage': 12}
            self.particles.emit(item.x + ITEM_SIZE // 2, item.y + ITEM_SIZE // 2, (200,
            200, 250), 20)
            self.items.remove(item)

def draw_ui(self):
    pygame.draw.rect(self.screen, COLORS['UI'], (0, 0, SCREEN_WIDTH, 80))
    pygame.draw.rect(self.screen, COLORS['TEXT_DARK'], (0, 0,
    SCREEN_WIDTH, 80), 3)

    hp_percent = self.player.hp / self.player.max_hp
    pygame.draw.rect(self.screen, COLORS['HEALTH_BG'], (20, 20, 300, 25))
    pygame.draw.rect(self.screen, COLORS['HEALTH'], (20, 20, 300 * hp_percent,
    25))

    exp_percent = self.player.exp / self.player.exp_to_next if self.player.exp_to_next
    > 0 else 0
    pygame.draw.rect(self.screen, COLORS['EXP_BG'], (20, 55, 300, 10))

```

```
pygame.draw.rect(self.screen, COLORS['EXP'], (20, 55, 300 * exp_percent, 10))
```

```
font = pygame.font.Font(None, 24)
```

```
big_font = pygame.font.Font(None, 28)
```

```
hp_text = font.render(f'HP: {self.player.hp}/{self.player.max_hp}', True,  
COLORS['TEXT'])
```

```
self.screen.blit(hp_text, (20, 18))
```

```
level_text = font.render(f'УРОВЕНЬ
```

```

{self.player.level}", True, COLORS['TEXT'])
self.screen.blit(level_text, (SCREEN_WIDTH - 250, 15))

gold_text = big_font.render(f" {self.player.gold}", True, COLORS['TEXT'])
self.screen.blit(gold_text, (SCREEN_WIDTH - 220, 40))

potion_text = big_font.render(f" {self.player.potions}", True, COLORS['TEXT'])
self.screen.blit(potion_text, (SCREEN_WIDTH - 220, 65))

enemies_text = font.render(f" Врагов осталось: {len(self.enemies)}", True,
COLORS['TEXT'])
self.screen.blit(enemies_text, (SCREEN_WIDTH // 2 - 100, 10))

if self.show_inventory:
    self.draw_inventory()

hint_font = pygame.font.Font(None, 18)
hints = ["WASD/Стрелки - движение", "Е - взаимодействие", "I - инвентарь",
"ESC - выход"]
for i, hint in enumerate(hints):
    hint_text = hint_font.render(hint, True, COLORS['TEXT_DARK'])
    self.screen.blit(hint_text, (10, SCREEN_HEIGHT - 80 + i * 20))

def draw_inventory(self):
    inv_surface = pygame.Surface((450, 550))
    inv_surface.fill(COLORS['UI'])
    pygame.draw.rect(inv_surface, COLORS['TEXT_DARK'],
    inv_surface.get_rect(), 4)

    font = pygame.font.Font(None, 36)

```



```

title = font.render("ИНВЕНТАРЬ", True, COLORS['EXP'])
inv_surface.blit(title, (150, 30))

y = 90
items_data = [
    (" Зелья здоровья", self.player.potions),
    (" Золото", self.player.gold),
    (" Оружие", self.player.equipment['weapon']['name'] if
self.player.equipment['weapon'] else "Нет")
]

for item_name, value in items_data:
    item_text = font.render(f"{item_name}: {value}", True, COLORS['TEXT'])
    inv_surface.blit(item_text, (30, y))
    y += 60

stats = [f" Сила: {self.player.strength}",
    f" Защита: {self.player.defense}",
    f" Урон: {self.player.get_damage()}"]
y += 40
for stat in stats:
    stat_text = pygame.font.Font(None, 28).render(stat, True,
COLORS['TEXT_DARK'])
    inv_surface.blit(stat_text, (30, y))
    y += 40

hint = pygame.font.Font(None, 24).render("Нажми I для закрытия", True,
COLORS['TEXT_DARK'])
inv_surface.blit(hint, (120, 500))

```

```
self.screen.blit(inv_surface, (SCREEN_WIDTH // 2 - 225, SCREEN_HEIGHT //  
2 - 275))
```

```
def draw(self):
```

```
self.screen.blit(self.background, (0, 0))
```

```
for item in self.items:
```

```
self.screen.blit(item.sprite, (item.x, item.y))
```

```
for enemy in self.enemies:
```

```
self.screen.blit(enemy.sprite, (enemy.x, enemy.y))
```

```
hp_percent = enemy.hp / enemy.max_hp
```

```
pygame.draw.rect(self.screen, COLORS['HEALTH_BG'], (enemy.x, enemy.y -  
10, SPRITE_SIZE_MAP, 6))
```

```
pygame.draw.rect(self.screen, COLORS['HEALTH'], (enemy.x, enemy.y - 10,  
SPRITE_SIZE_MAP * hp_percent, 6))
```

```
for npc in self.npcs:
```

```
self.screen.blit(npc.sprite, (npc.x, npc.y))
```

```
font = pygame.font.Font(None, 16)
```

```
name_text = font.render(npc.id.upper(), True, COLORS['TEXT'])
```

```
self.screen.blit(name_text, (npc.x + 8, npc.y - 15))
```

```
self.screen.blit(self.player.sprite, (self.player.x, self.player.y))
```

```
self.particles.draw(self.screen)
```

```
self.draw_ui()
```

```
if self.combat:
```

```
self.combat.draw()
```

```
if self.dialogue:  
    self.dialogue.draw()
```

```
if self.victory:  
    self.victory_screen.draw()
```

```
if self.defeat:  
    self.defeat_screen.draw()
```

```
pygame.display.flip()
```

```
def run(self):  
    while self.running:  
        self.dt = self.clock.tick(FPS) / 1000.0  
        self.handle_events()  
        self.update()  
        self.draw()
```

```
pygame.quit()  
sys.exit()
```

```
if __name__ == "__main__":  
    game = Game()  
    game.run\(\)
```

